

Proyecto Final

Optimizacion de Rutas

con Algoritmo Genetico

Python · React · Firebase · Google Maps API

**SI SE ENCUENTRA UNA API KEY EN EL REPOSITORIO,
LA CALIFICACION ES AUTOMATICAMENTE 0.**

Descripcion general. Construir una aplicacion web completa que, dados hasta 15 destinos dentro de un radio maximo de 100 km entre si, calcule y visualice la ruta optima utilizando un algoritmo genetico. El calculo ocurre en una Cloud Function; el frontend muestra la ruta en un mapa interactivo con un pin por cada destino.

Modalidad

Trabajo en parejas. Ambos integrantes deben poder explicar cualquier parte del proyecto en la entrega. Si algo no queda claro o falta un requerimiento, pregunten antes de construir.

Lenguajes y herramientas obligatorias

- **Backend:** Python con uv para entornos virtuales y dependencias.
- **Frontend:** React + Vite.
- **Autenticacion:** Firebase Authentication.
- **Funcion en la nube:** Firebase Cloud Functions o GCP Cloud Functions (Python).
- **Mapa:** Google Maps JavaScript API.
- **Diagramas:** draw.io con iconos oficiales de Google Cloud Platform.

Requerimientos funcionales

Ingreso de destinos

- Entre 2 y 15 destinos por calculo.
- Todos dentro de un radio maximo de 100 km entre si; la aplicacion debe validar esto y notificar si se viola.
- El usuario elige el modo de calculo:
 - **Ruta cerrada:** regresa al punto de origen.
 - **Ruta abierta:** termina en el ultimo destino.

Algoritmo genetico en la nube

- La Cloud Function recibe los destinos y el modo de calculo.
- Construye la matriz de distancias con la API de Google Maps Distance Matrix.
- Ejecuta el algoritmo genetico y retorna el orden optimo y la distancia total.
- La funcion solo acepta llamadas desde las IPs configuradas por el alumno en GCP. Cualquier otra IP debe ser rechazada.

Visualizacion del resultado

- Mapa interactivo con la ruta trazada y un pin numerado por destino.
- Distancia total del recorrido visible en pantalla.

Autenticacion

- Acceso a la aplicacion solo con login via Firebase Authentication.
- Solo usuarios autenticados pueden invocar el calculo.

Responsividad

La interfaz debe funcionar correctamente tanto en navegador de escritorio como en dispositivos moviles.

Requerimientos de entrega

Repositorio publico en GitHub

- **Commits incrementales** que reflejen el progreso real: frontend, backend, correcciones, mejoras. Un repositorio con un solo commit o sin historial coherente penaliza significativamente. Estamos formando ingenieros con buenas practicas.
- **SI SE ENCUENTRA UNA API KEY O CREDENCIAL EN CUALQUIER ARCHIVO DEL REPOSITORIO, LA CALIFICACION ES AUTOMATICAMENTE 0.**
- Credenciales manejadas con variables de entorno o Secret Manager. Los archivos `.env` en `.gitignore` siempre.

README en el repositorio

- Descripcion general y arquitectura del proyecto.
- Estructura de carpetas documentada.
- Instrucciones paso a paso para correr el proyecto desde cero en una maquina nueva: clonar, instalar dependencias, configurar variables de entorno, correr backend y frontend.
- Archivos de dependencias presentes: `pyproject.toml` y `uv.lock` para Python; `package.json` para el frontend.

Diagramas (entrega separada)

- **Diagrama de flujo** del usuario: desde el login hasta la visualizacion de la ruta.
- **Diagrama de arquitectura** del sistema completo en draw.io con iconos oficiales de Google Cloud Platform.
- Entregados como `.drawio` y exportados en PDF o PNG.

Estructura sugerida del repositorio

Sugerencia de organizacion. Si el equipo tiene una razon tecnica para hacerlo diferente, documentenlo en el README.

```
route-optimizer/  
  README.md  
  .gitignore  
  
  backend/  
    pyproject.toml          # dependencias Python (uv)  
    uv.lock  
    main.py                 # entry point de la Cloud Function  
    genetic_algorithm.py    # logica del algoritmo genetico  
    distance_matrix.py      # llamadas a Distance Matrix API  
    .env.example            # variables requeridas, sin valores  
    reales  
  
  frontend/  
    package.json  
    vite.config.js  
    index.html  
    src/  
      main.jsx  
      App.jsx  
      components/  
        Map.jsx  
        DestinationInput.jsx  
        RouteResult.jsx  
      services/  
        firebase.js         # configuracion Firebase Auth  
        cloudFunction.js    # llamada a la Cloud Function  
      .env.example  
  
  diagrams/  
    flow.drawio  
    architecture.drawio
```

Referencias: [uv](#) · [Firebase Auth](#) · [Maps JS API](#) · [Distance Matrix API](#) · [Cloud Functions Python](#) · [draw.io](#)

Rubrica de evaluacion

Cada criterio se califica de forma independiente sobre 100. Los marcados con ★ pueden anular la nota del proyecto si no se cumplen.

Criterio	Descripcion	Nota
rowgray	Funcionalidad	
Ingreso de destinos	2–15 destinos, radio 100 km, validacion y notificacion	/ 100
rowgray Modos de calculo	Ruta cerrada y ruta abierta funcionando correctamente	/ 100
Algoritmo genetico	Correcto sobre la matriz de distancias de Google Maps	/ 100
rowgray Visualizacion	Mapa con pines numerados y distancia total visible	/ 100
rowgray	Arquitectura y seguridad	
Cloud Function	Desplegada, funcional e invocable desde el frontend	/ 100
rowgray Restriccion de IP	Configurada; rechaza llamadas desde otras IPs	/ 100
Autenticacion	Login con Firebase; calculo bloqueado sin sesion	/ 100
rowgray Credenciales ★	Cero API keys o secretos expuestos en el repositorio	/ 100
rowgray	Calidad del codigo	
Commits ★	Historial incremental coherente con el progreso real	/ 100
rowgray Documentacion	Codigo legible, comentado y README funcional	/ 100
Dependencias	pyproject.toml, uv.lock y package.json presentes	/ 100
rowgray	Diagramas	
Flujo de usuario	Desde login hasta visualizacion de la ruta	/ 100
rowgray Arquitectura	draw.io con iconos oficiales de GCP	/ 100
rowgray	Estetica y UX	
Interfaz	Limpia, consistente y responsiva en mobile y desktop	/ 100
rowgray	Ronda de preguntas	
Dominio tecnico	Preguntas arbitrarias sobre cualquier aspecto del proyecto	/ 100

★ Credenciales en el repositorio resulta en nota 0 automatica. Ausencia de commits incrementales afecta directamente ese criterio.

Ronda de preguntas

Como parte de la entrega, cada pareja pasara por una **ronda de preguntas en vivo**. Las preguntas son arbitrarias: pueden cubrir cualquier decision tecnica, cualquier linea de codigo, cualquier componente de la arquitectura o cualquier herramienta utilizada. No hay lista predefinida.

Que se evalua

- Que ambos integrantes comprenden el proyecto completo, no solo la parte que cada quien construyo.
 - Capacidad de explicar decisiones de diseño y arquitectura con criterio.
 - Comprension del algoritmo genetico: cromosoma, fitness, cruce, mutacion, criterio de parada.
 - Entendimiento del flujo de seguridad: autenticacion, restriccion de IP, manejo de credenciales.
 - Dominio del stack: como funciona cada pieza y por que se eligio.
-

Como prepararse

No existe preparacion especifica mas alla de **haber construido y entendido cada parte del proyecto**. Leer el codigo del companero, revisar la arquitectura juntos y poder trazar el flujo completo de una peticion de principio a fin son los mejores indicadores de que estan listos.

Esta ronda se califica de forma independiente sobre 100, igual que los demas criterios de la rubrica. Ambos integrantes pueden recibir notas distintas segun sus respuestas individuales.